Project Python Foundations: FoodHub Data Analysis

Marks: 60

Context

The number of restaurants in New York is increasing day by day. Lots of students and busy professionals rely on those restaurants due to their hectic lifestyles. Online food delivery service is a great option for them. It provides them with good food from their favorite restaurants. A food aggregator company FoodHub offers access to multiple restaurants through a single smartphone app.

The app allows the restaurants to receive a direct online order from a customer. The app assigns a delivery person from the company to pick up the order after it is confirmed by the restaurant. The delivery person then uses the map to reach the restaurant and waits for the food package. Once the food package is handed over to the delivery person, he/she confirms the pick-up in the app and travels to the customer's location to deliver the food. The delivery person confirms the drop-off in the app after delivering the food package to the customer. The customer can rate the order in the app. The food aggregator earns money by collecting a fixed margin of the delivery order from the restaurants.

Objective

The food aggregator company has stored the data of the different orders made by the registered customers in their online portal. They want to analyze the data to get a fair idea about the demand of different restaurants which will help them in enhancing their customer experience. Suppose you are hired as a Data Scientist in this company and the Data Science team has shared some of the key questions that need to be answered. Perform the data analysis to find answers to these questions that will help the company to improve the business.

Data Description

The data contains the different data related to a food order. The detailed data dictionary is given below.

Data Dictionary

- order_id: Unique ID of the order
- customer_id: ID of the customer who ordered the food
- restaurant_name: Name of the restaurant
- cuisine_type: Cuisine ordered by the customer

- cost: Cost of the order
- day_of_the_week: Indicates whether the order is placed on a weekday or weekend (The weekday is from Monday to Friday and the weekend is Saturday and Sunday)
- rating: Rating given by the customer out of 5
- food_preparation_time: Time (in minutes) taken by the restaurant to prepare the food. This is calculated by taking the difference between the timestamps of the restaurant's order confirmation and the delivery person's pick-up confirmation.
- delivery_time: Time (in minutes) taken by the delivery person to deliver the food package. This is calculated by taking the difference between the timestamps of the delivery person's pick-up confirmation and drop-off information

Let us start by importing the required libraries

In [198]:

```
# Installing the libraries with the specified version.
!pip install numpy==2.0.2 pandas==2.2.2 matplotlib==3.10.0 seaborn==0.13.2 -q
--user
```

In [199]:

```
# import libraries for data manipulation
import numpy as np
import pandas as pd

# import libraries for data visualization
import matplotlib.pyplot as plt
import seaborn as sns
```

Understanding the structure of the data

In [200]:

```
# uncomment and run the following lines for Google Colab
# from google.colab import drive
# drive.mount('/content/drive')
```

In [201]:

```
# read the data
df = pd.read_csv('foodhub_order.csv')
# returns the first 5 rows
df.head()
```

Out[201]:

	order_id	customer_id	restaurant_name	cuisine_type	cost_of_the_order	day_of_the_week	rating	food
0	1477147	337525	Hangawi	Korean	30.75	Weekend	Not given	25
1	1477685	358141	Blue Ribbon Sushi Izakaya	Japanese	12.08	Weekend	Not given	25
2	1477070	66393	Cafe Habana	Mexican	12.23	Weekday	5	23

	order_id	customer_id	restaurant_name	cuisine_type	cost_of_the_order	day_of_the_week	rating	food
3	1477334	106968	Blue Ribbon Fried Chicken	American	29.20	Weekend	3	25
4	1478249	76942	Dirty Bird to Go	American	11.59	Weekday	4	25

Observations:

The DataFrame has 9 columns as mentioned in the Data Dictionary. Data in each row corresponds to the order placed by a customer.

Question 1: How many rows and columns are present in the data? [0.5 mark]

In [202]:

```
# Write your code here
df.shape
```

Out[202]:

```
(1898, 9)
```

Observations: Shape of the Dataset

- The dataset contains 1898 rows and 9 attribute columns .
- Each row corresponds to one customer order.

Question 2: What are the datatypes of the different columns in the dataset? (The info() function can be used) [0.5 mark]

In [203]:

```
# Use info() to print a concise summary of the DataFrame
df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1898 entries, 0 to 1897
Data columns (total 9 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   order_id               1898 non-null   int64
 1   customer_id            1898 non-null   int64
 2   restaurant_name        1898 non-null   object
 3   cuisine_type           1898 non-null   object
 4   cost_of_the_order      1898 non-null   float64
 5   day_of_the_week        1898 non-null   object
 6   rating                 1898 non-null   object
 7   food_preparation_time  1898 non-null   int64
 8   delivery_time          1898 non-null   int64
```

```
dtypes: float64(1), int64(4), object(4)
memory usage: 133.6+ KB
```

Observations:

- Numerical columns: cost_of_the_order, food_preparation_time,delivery_time
- Categorical columns: restaurant_name, cuisine_type, day_of_the_week, rating
- The 'rating' column is of type object because it includes the value "Not given"

Question 3: Are there any missing values in the data? If yes, treat them using an appropriate method. [1 mark]

In [204]:

```
# Write your code here
df.isnull().sum()
```

Out[204]:

	0
order_id	0
customer_id	0
restaurant_name	0
cuisine_type	0
cost_of_the_order	0
day_of_the_week	0
rating	0
food_preparation_time	0
delivery_time	0

dtype: int64

Observations:

Missing Values

- There are 'no missing (null) values' in the dataset.
- However, the 'rating' column includes "Not given", which represents customers who chose not to rate their order.
- This is not missing data but a meaningful category reflecting customer behavior.

Question 4: Check the statistical summary of the data. What is the minimum, average, and maximum time it takes for food to be prepared once an order is placed? [2 marks]

In [205]:

```
# Write your code here
df.describe().T
```

Out[205]:

	count	mean	std	min	25%	50%	
order_id	1898.0	1.477496e+06	548.049724	1476547.00	1477021.25	1477495.50	1.47797
customer_id	1898.0	1.711685e+05	113698.139743	1311.00	77787.75	128600.00	2.70525
cost_of_the_order	1898.0	1.649885e+01	7.483812	4.47	12.08	14.14	2.22975
food_preparation_time	1898.0	2.737197e+01	4.632481	20.00	23.00	27.00	3.10000
delivery_time	1898.0	2.416175e+01	4.972637	15.00	20.00	25.00	2.80000

Observations:

- Minimum food preparation time: 20 minutes
- Average food preparation time: 27.37 minutes
- Maximum food preparation time: 35 minutes

Interpretation:

- Most restaurants take around 27 minutes on average to prepare food.
- The preparation time ranges from 20 to 35 minutes, indicating relatively consistent performance across restaurants.
- The small range suggests that most restaurants operate within a similar preparation time window, with limited extreme delays.

Question 5: How many orders are not rated? [1 mark]

In [206]:

```
# Rating distribution


df['rating'].value_counts()
```

Out[206]:

	count
rating	
Not given	736
5	588
4	386
3	188

dtype: int64

In [207]:

```
df['rating'].value_counts(normalize=True)
```

Out[207]:

	proportion
rating	
Not given	0.387777
5	0.309800
4	0.203372
3	0.099052

dtype: float64

Observations:

The number of orders with no rating ("Not given") is "736".

Interpretation:

- This represents approximately **38.8%** of all orders.
- A significant portion of customers are not providing feedback.
- This suggests an opportunity for the company to improve customer engagement by encouraging ratings through reminders or incentives.

Exploratory Data Analysis (EDA)

Univariate Analysis

Question 6: Explore all the variables and provide observations on their distributions. (Generally, histograms, boxplots, countplots, etc. are used for univariate exploration.) [9 marks]

In [208]:

```
# check unique order ID
df['order_id'].nunique()
```

Out[208]:

```
1898
```

Observations: there are 1898 unique id with no repations

In [209]:

```
# check customer ID to know how many different people ordered food
df['customer_id'].nunique()
```

Out[209]:

```
1200
```

In [210]:

```
# check customer ID to know average odering per customer
df.shape[0]/df['customer_id'].nunique()
```

Out[210]:

```
1.5816666666666668
```

The number of unique customers was calculated using the 'nunique()' function on the 'customer_id' column.

Observation:

- The dataset contains a specific number of 'unique customers' (as obtained from the output).
- Since the total number of orders is higher than the number of unique customers, this indicates that some customers have placed multiple orders.
- This suggests the presence of 'repeat customers', which is a positive sign for customer retention and platform loyalty.
- On average, each customer places multiple orders, indicating good customer retention.

In [211]:

```
# check Unique resturant name
df['restaurant_name'].nunique()
```

Out[211]:

```
178
```

In [212]:

```
df['restaurant_name'].value_counts()
```

Out[212]:

	count
restaurant_name	
Shake Shack	219
The Meatball Shop	132
Blue Ribbon Sushi	119
Blue Ribbon Fried Chicken	96
Parm	68
...	...
Rye House	1
Hiroko's Place	1
Frank Restaurant	1
Sarabeth's West	1
'wichcraft	1

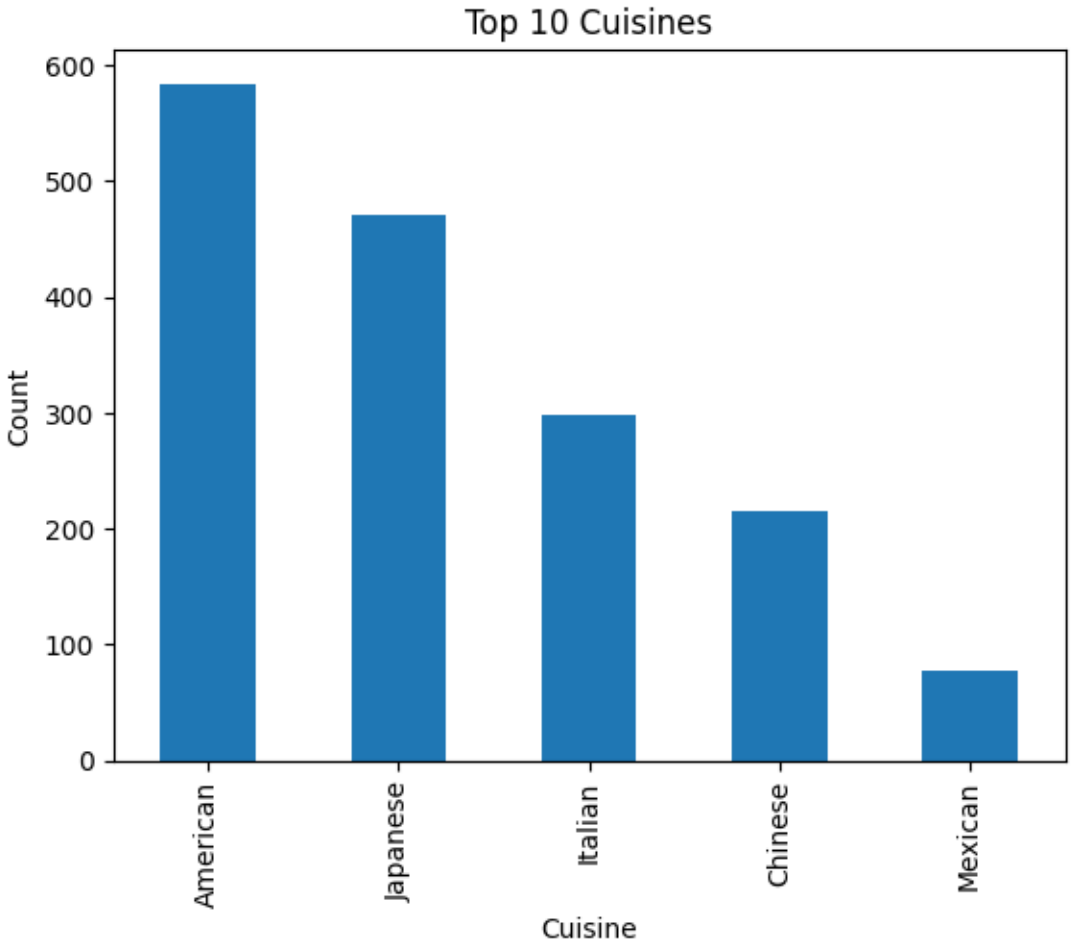
178 rows × 1 columns

dtype: int64

In this section, we explore each variable individually using appropriate visualizations such as histograms and count plots to understand their distribution.

In [213]:

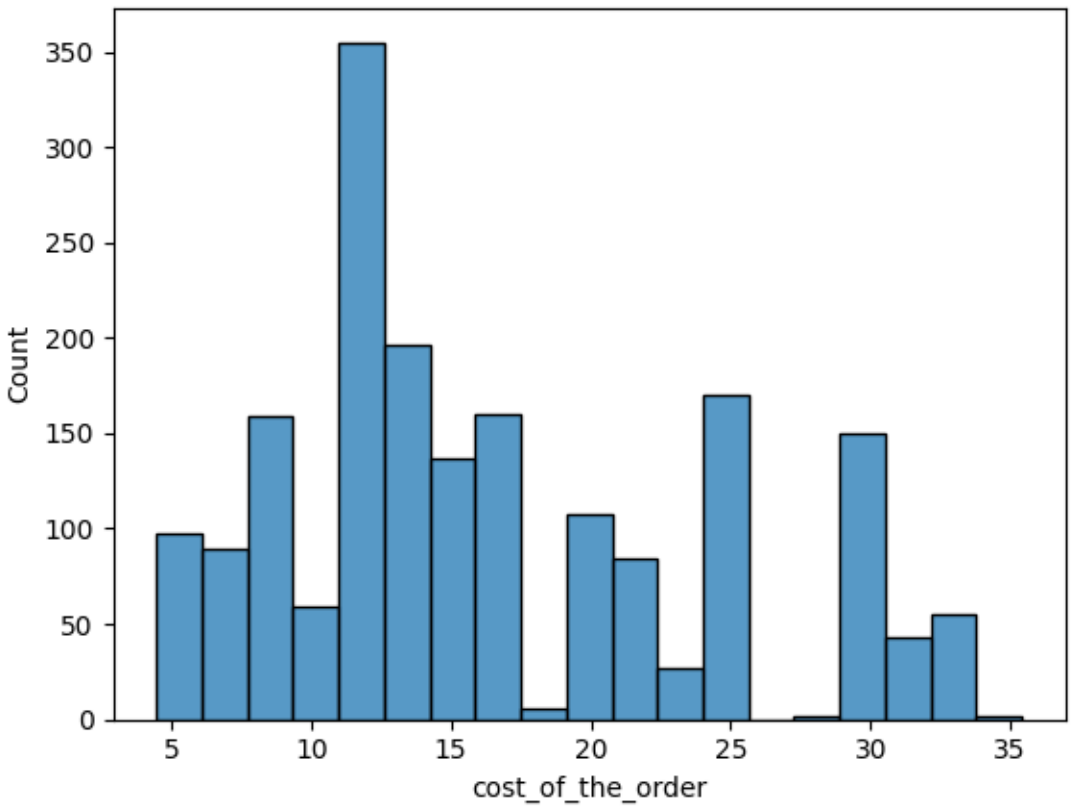
```
# 3. "What are the most popular cuisines on the platform?"
# find the top 10 most frequent categories and visualize them
# Cuisine = categorical data
# Bar charts are best for comparing categories
df['cuisine_type'].value_counts().head().plot(kind='bar',)
plt.title('Top 10 Cuisines')
plt.xlabel('Cuisine')
plt.ylabel('Count')
plt.show()
```

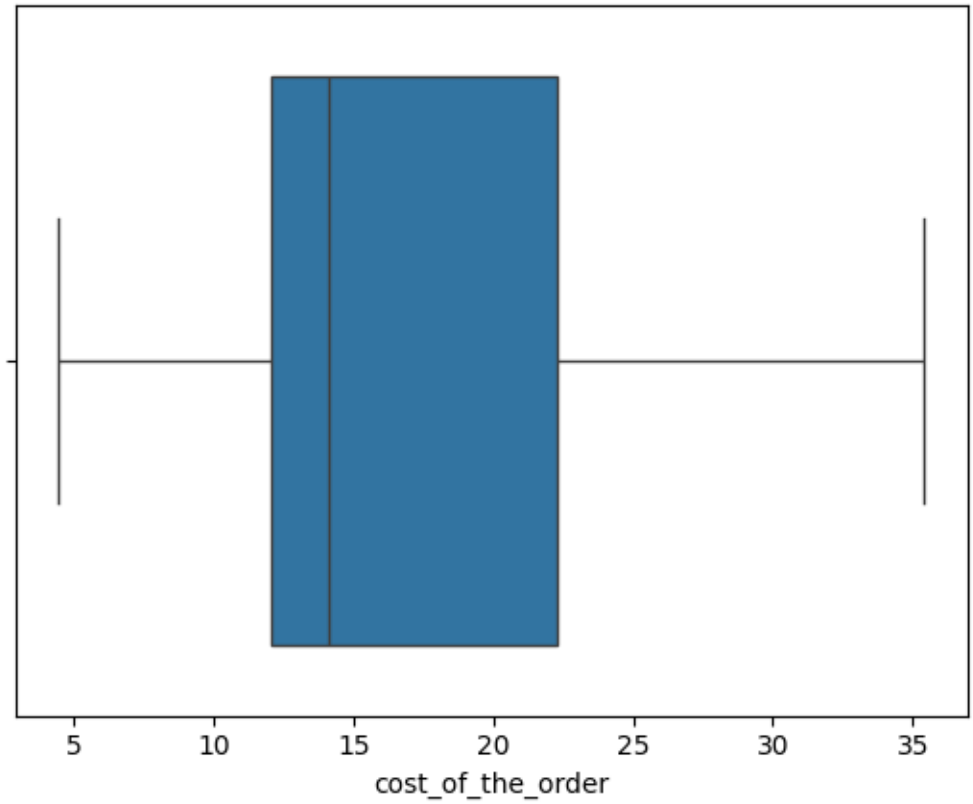


Observation: The top cuisines dominate the majority of orders, indicating strong customer preferences. These cuisines should be prioritized for promotions and partnerships.

In [214]:

```
sns.histplot(data = df, x = 'cost_of_the_order')
plt.show()
sns.boxplot(data = df, x = 'cost_of_the_order')
plt.show()
```



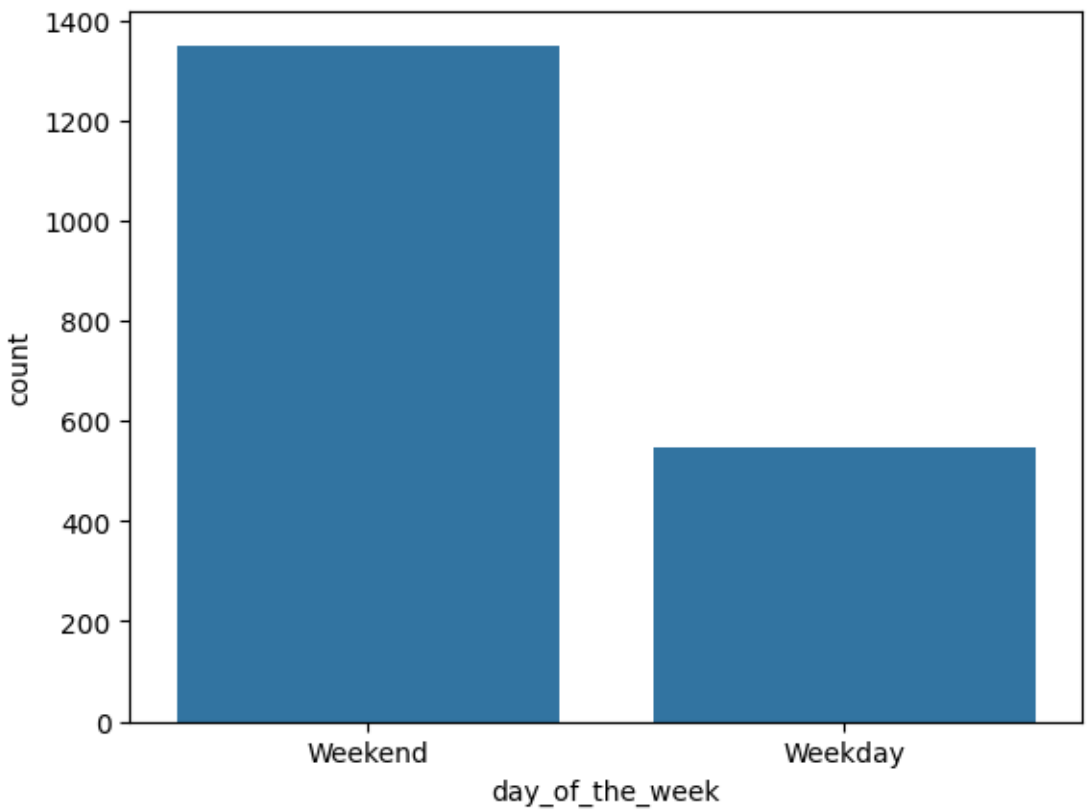


Observation: Cost of the Order

- The distribution of order cost is slightly **right-skewed**, indicating that most customers place lower to mid-range orders, with fewer high-cost orders.
- The majority of orders fall within the range of approximately **$12 to $22**, suggesting a common spending pattern among customers.
- The boxplot shows that the median order cost lies around the central range, indicating a balanced distribution of typical spending.
- A few higher-cost orders (above **$30**) are present, which appear as potential **outliers**, representing premium purchases.
- The presence of higher-cost outliers suggests an opportunity for the business to promote premium offerings to increase revenue.
- Overall, the cost distribution is relatively consistent, with limited extreme variation, indicating stable customer spending behavior.

In [215]:

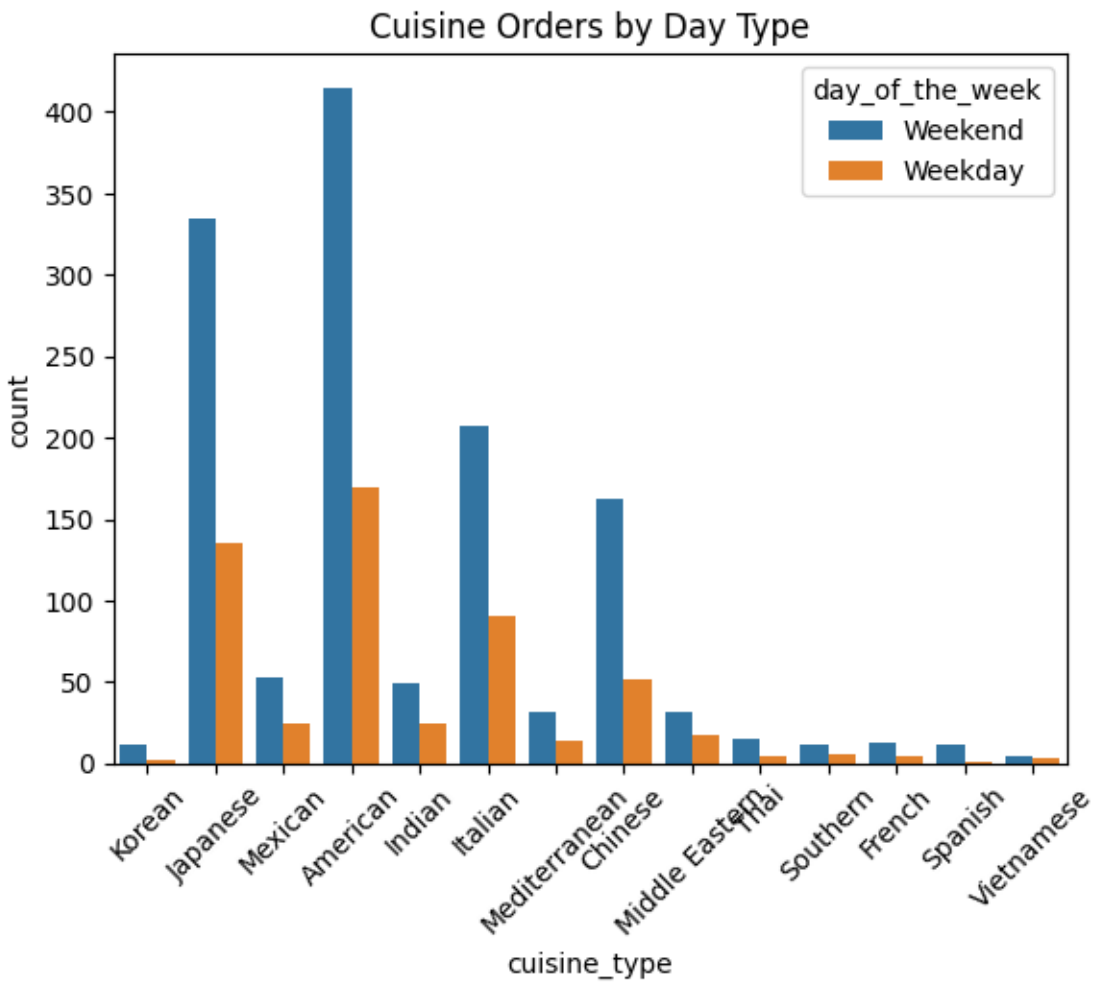
```
sns.countplot(data = df, x = 'day_of_the_week')
plt.show()
```



Observation: The countplot shows the distribution of orders across weekdays and weekends. The number of orders varies between the two categories, indicating differences in customer ordering behavior depending on the day. Typically, higher order volumes are expected on weekends due to increased leisure time.

In [216]:

```
# Cuisine vs Day of Week
sns.countplot(data=df, x='cuisine_type', hue='day_of_the_week')
plt.xticks(rotation=45)
plt.title("Cuisine Orders by Day Type")
plt.show()
```



Observantion: The distribution of cuisines varies between weekdays and weekends. Certain cuisines appear to be more popular on weekends, possibly due to changes in customer preferences during leisure time.

In [217]:

```
# check the unique values of the Rating
df['rating'].value_counts()
```

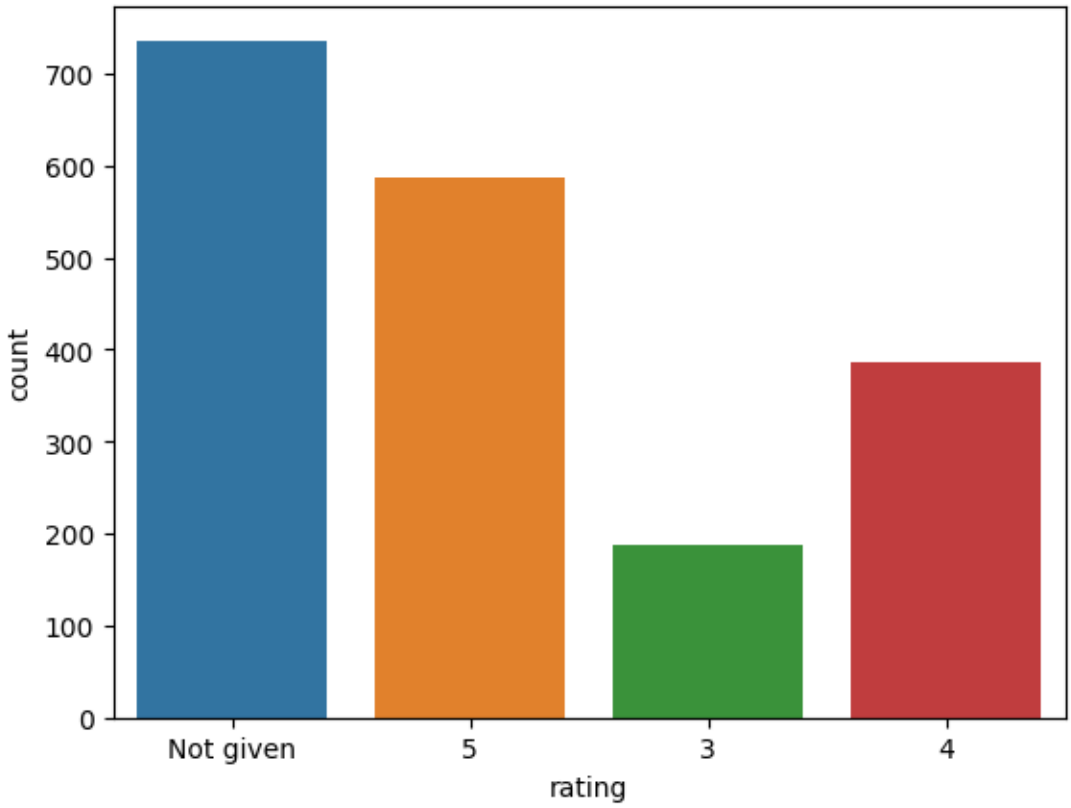
Out[217]:

	count
rating	
Not given	736
5	588
4	386
3	188

dtype: int64

In [218]:

```
# 2. Rating
# Count how many times each rating occurs and display the results as a bar
chart.”
sns.countplot(data = df, x = 'rating', hue= 'rating')
plt.show()
```

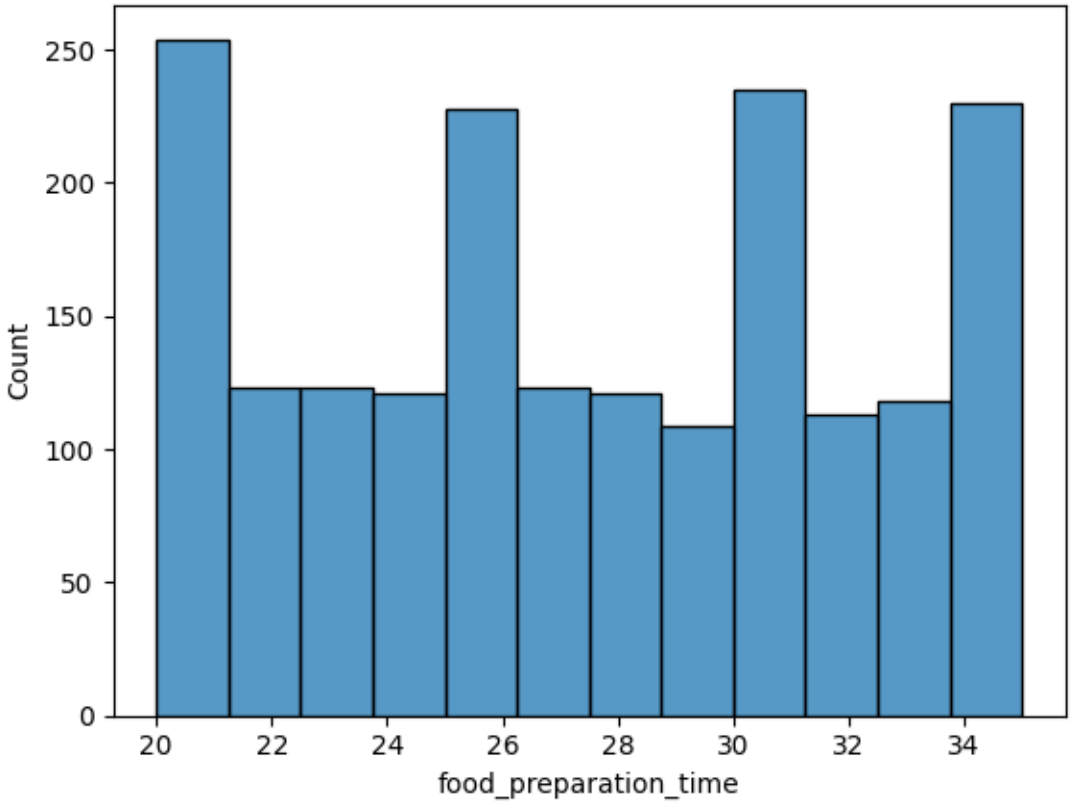


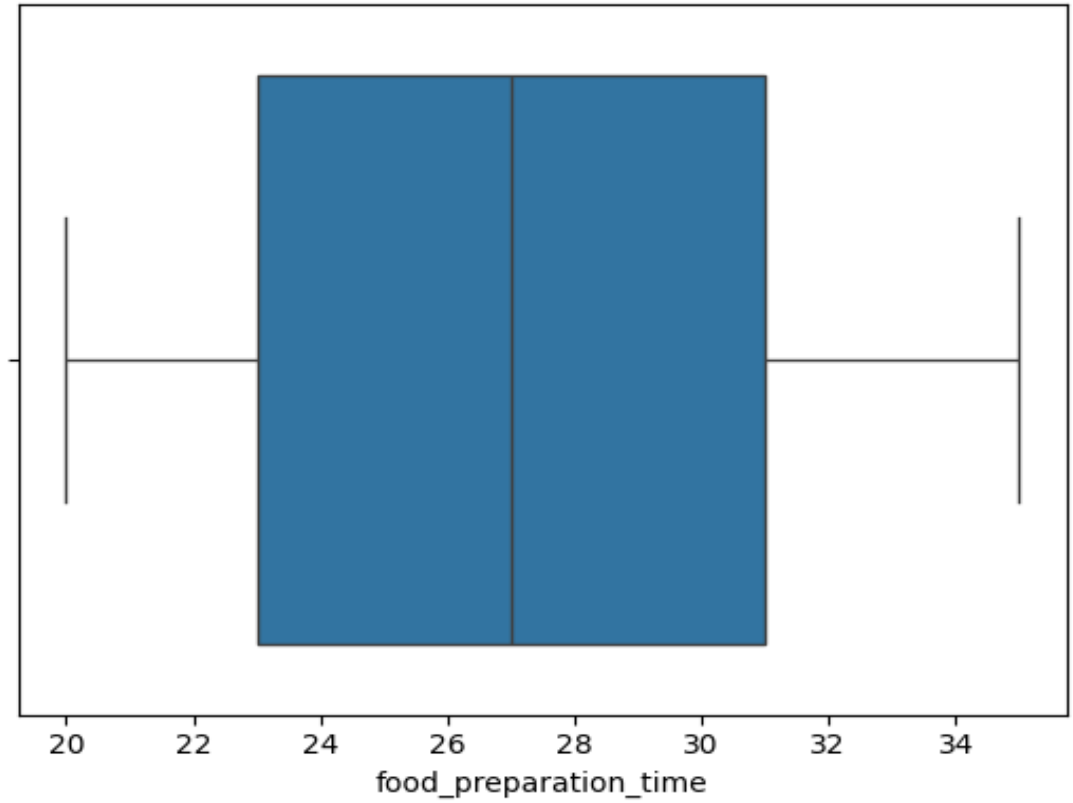
Observation:

- Ratings range from 3 to 5, with no ratings below 3.
- The majority of ratings are 4 and 5, indicating high customer satisfaction.
- A large proportion (38.8%) of ratings are "Not given", indicating low customer engagement.

In [219]:

```
# food preparation time
sns.histplot(data=df, x= 'food_preparation_time')
plt.show()
sns.boxplot(data=df, x = 'food_preparation_time')
plt.show()
```


Count
0
50
100
150
200
250
20
22
24
26
28
30
32
34
food_preparation_time


20
22
24
26
28
30
32
34
food_preparation_time

Observation: Food Preparation Time

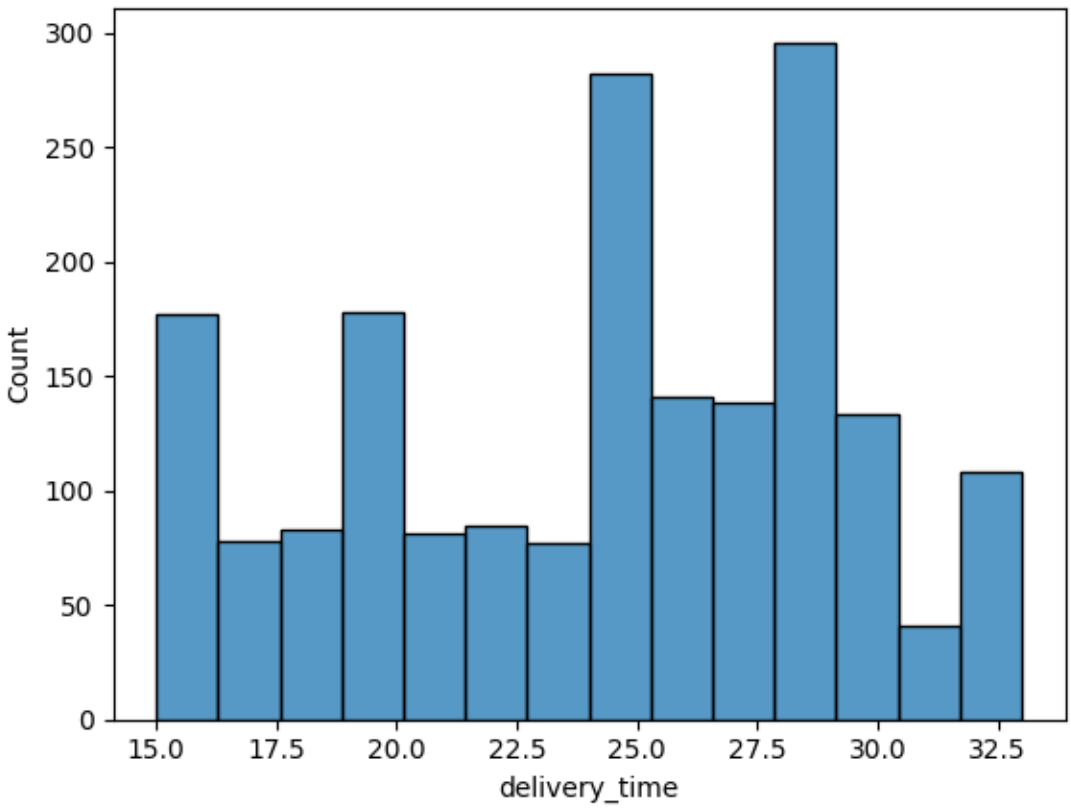
The food preparation time ranges between approximately **20 and 35 minutes**, with most orders being prepared within the **25 to 30 minute range**. The distribution appears fairly consistent, indicating that restaurants follow a similar preparation time pattern.

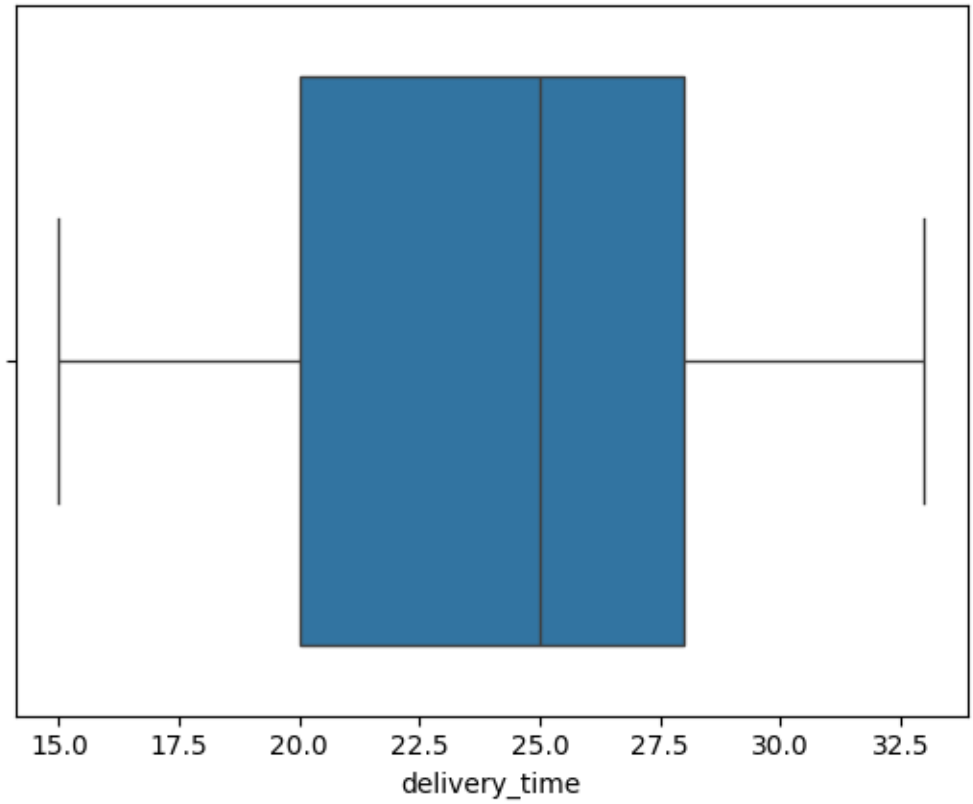
The boxplot shows that the median preparation time lies around the central range, suggesting balanced performance across restaurants. There are no significant outliers, indicating that extreme delays in preparation are uncommon.

Overall, the preparation time is stable and predictable, which contributes positively to customer experience.

In [220]:

```
# Delivery Time
sns.histplot(data=df, x= 'delivery_time')
plt.show()
sns.boxplot(data=df, x = 'delivery_time')
plt.show()
```





Observation: Delivery Time

The delivery time ranges from approximately **15 to 33 minutes**, with most deliveries occurring between **20 and 28 minutes**. The histogram shows that delivery times are fairly concentrated within this range, indicating consistent delivery performance.

The boxplot is left skwed and indicates that the median delivery time lies around the mid-range, suggesting that most deliveries are completed within a reasonable timeframe. There are no significant outliers, implying that extreme delays are uncommon.

Overall, the delivery process appears to be efficient and stable, which is important for maintaining customer satisfaction.

Question 7: Which are the top 5 restaurants in terms of the number of orders received? [1 mark]

In [221]:

```
# Top 5 Restaurants
df['restaurant_name'].value_counts(normalize=True).head(5)
```

Out[221]:

	proportion
restaurant_name	
Shake Shack	0.115385
The Meatball Shop	0.069547
Blue Ribbon Sushi	0.062698
Blue Ribbon Fried Chicken	0.050580
Parm	0.035827

dtype: float64

Observations:The top 5 restaurants in terms of the number of orders received are:

(Shake shack, The meathball shop, Blue ribbonSushi,Blue Ribbon Fried Chicken,Pam)

Interpretation:

- These restaurants have the highest number of orders, indicating strong customer demand and popularity.
- They are key contributors to the platform's total orders.
- The company can prioritize these restaurants for promotions, partnerships, and improved service offerings to maximize revenue.

Question 8: Which is the most popular cuisine on weekends? [1 mark]

In [222]:

```
# The popular cuisine on weekend
df[df['day_of_the_week'] == 'Weekend']['cuisine_type'].value_counts()
```

Out[222]:

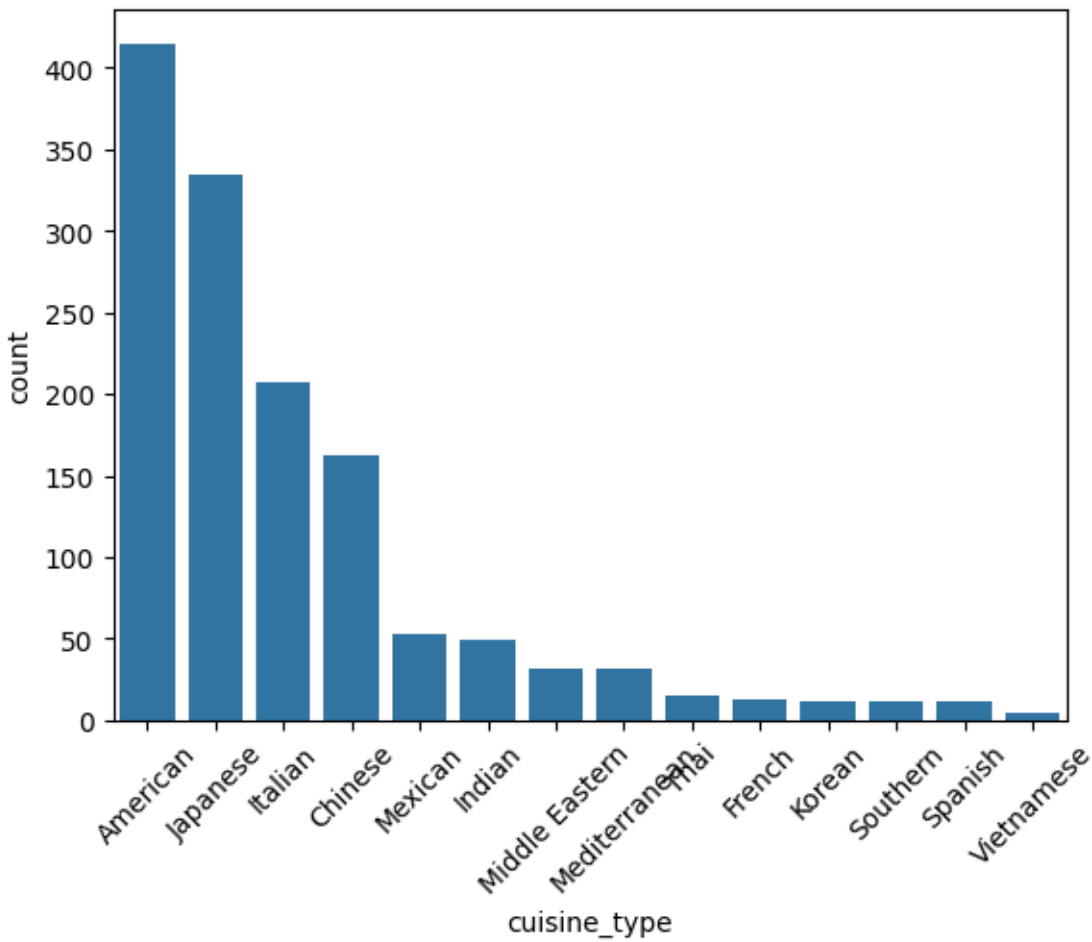
	count
cuisine_type	
American	415
Japanese	335
Italian	207
Chinese	163
Mexican	53
Indian	49
Middle Eastern	32
Mediterranean	32
Thai	15

	count
cuisine_type	
French	13
Korean	11
Southern	11
Spanish	11
Vietnamese	4

dtype: int64

In [223]:

```
sns.countplot(
    data=df[df['day_of_the_week'] == 'Weekend'],
    x='cuisine_type',
    order=df[df['day_of_the_week'] ==
'Weekend']['cuisine_type'].value_counts().index
)
plt.xticks(rotation=45)
plt.show()
```



Observations:The most popular cuisine on weekends is:

(American)

Interpretation:

- This cuisine has the highest number of orders during weekends, indicating strong customer preference.
- It may reflect weekend-specific eating habits or preferences for certain types of food.
- The company can promote this cuisine during weekends to maximize sales and customer satisfaction.

Question 9: What percentage of the orders cost more than 20 dollars? [2 marks]

In [224]:

```
# % of Order > $20
percentage = (df[df['cost_of_the_order'] > 20].shape[0] / df.shape[0]) * 100
percentage
```

Out[224]:

```
29.24130663856691
```

Observations:

The percentage of orders costing more than $20 is calculated as:

(Number of orders with cost > $20 / Total number of orders) × 100

The calculated percentage is:

(29.24)%

Interpretation:

- This represents the proportion of high-value orders on the platform.
- A higher percentage indicates that customers are willing to spend more, contributing significantly to revenue.
- These orders are important for the business as they generate higher commission earnings.

Question 10: What is the mean order delivery time? [1 mark]

In [225]:

```
# Mean order delivery time
```

```
df['delivery_time'].mean()
```

Out[225]:

```
np.float64(24.161749209694417)
```

Observations:

The mean (average) delivery time is approximately **24.16 minutes**.

Interpretation:

- On average, it takes about 24 minutes for an order to be delivered after pickup.
- This indicates relatively efficient delivery performance.
- Maintaining or reducing this time can help improve customer satisfaction.

Question 11: The company has decided to give 20% discount vouchers to the top 5 most frequent customers. Find the IDs of these customers and the number of orders they placed. [1 mark]

In [226]:

```
#  Top 5 Most Frequent Customers
df['customer_id'].value_counts().head(5)
```

Out[226]:

	count
customer_id	
52832	13
47440	10
83287	9
250494	8
259341	7

dtype: int64

The top 5 most frequent customers and the number of orders are as bove.

Interpretation:

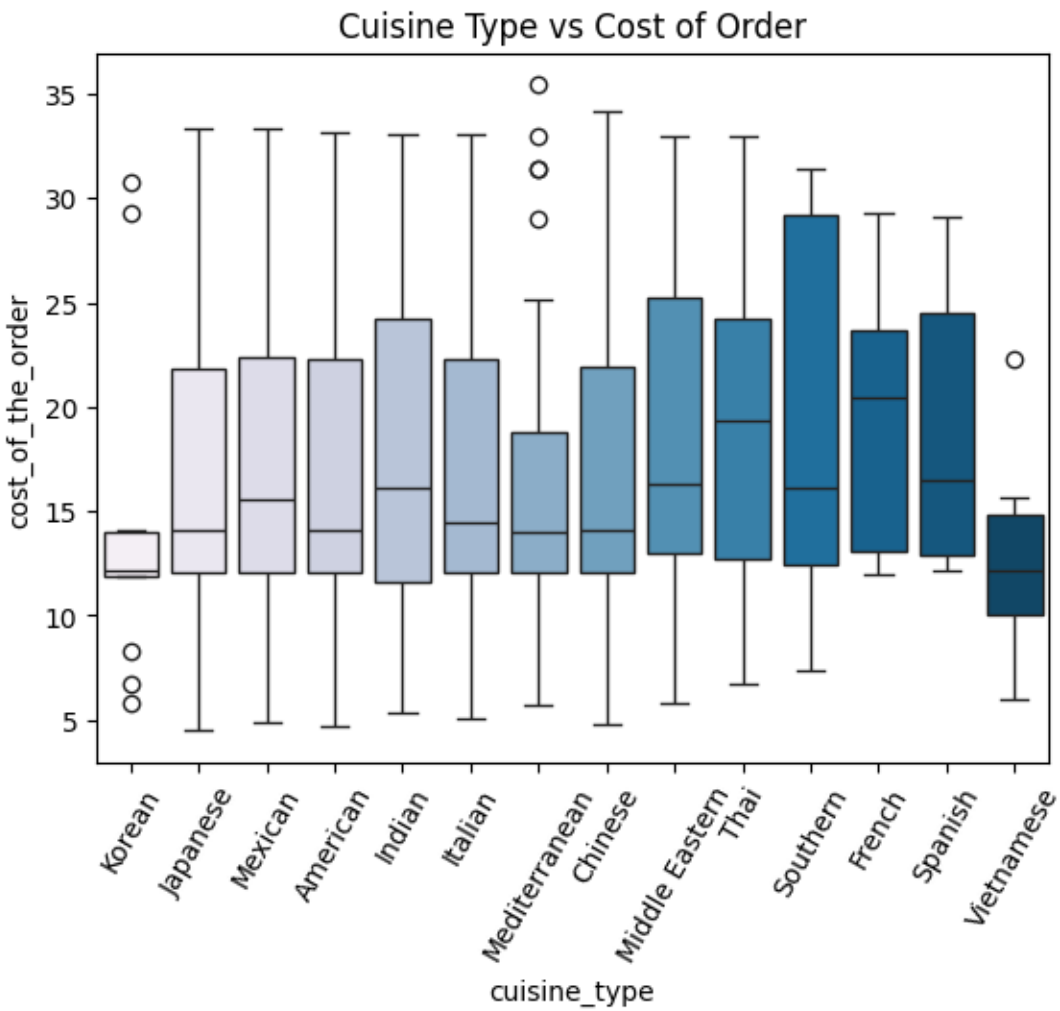
- These customers have placed the highest number of orders on the platform.
- They are highly valuable customers and contribute significantly to total orders.
- Offering them a 20% discount voucher is a strategic decision to retain loyal customers and encourage continued usage.

Multivariate Analysis

Question 12: Perform a multivariate analysis to explore relationships between the important variables in the dataset. (It is a good idea to explore relations between numerical variables as well as relations between numerical and categorical variables) [10 marks]

In [227]:

```
#  Cuisine Type (categorical) vs Cost of Order (numerical)
sns.boxplot(x="cuisine_type", y="cost_of_the_order", data =df,
palette='PuBu')
plt.xticks(rotation=60)
plt.title("Cuisine Type vs Cost of Order")
plt.show()
/tmp/ipykernel_4005/3607152041.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed
in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the
same effect.

  sns.boxplot(x="cuisine_type", y="cost_of_the_order", data =df,
palette='PuBu')
```



Observation: Cuisine Type vs Cost of Order

The cost of orders varies across different cuisine types, indicating differences in pricing and customer spending patterns. Some cuisines have higher median costs, suggesting they are generally more expensive, while others are more affordable.

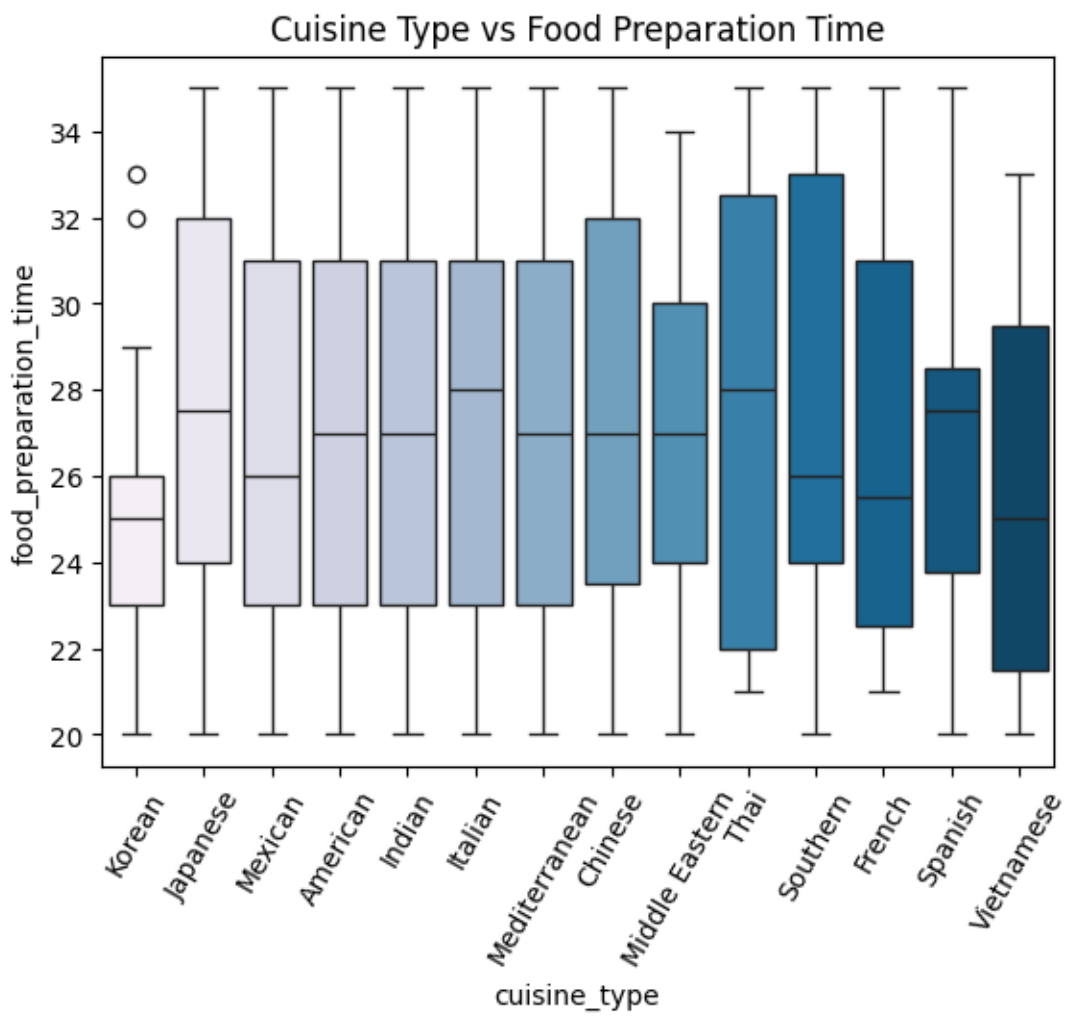
The spread of the boxplots shows that certain cuisines have a wider range of prices, indicating variability in order costs. Additionally, a few cuisines display higher-cost outliers, representing premium or large orders.

Overall, this suggests that customer spending behavior differs depending on the cuisine type, which can help the business tailor pricing strategies and promotional offers.

In [228]:

```
# Cuisine Type (categorical) vs Food Preparation Time (numerical)
sns.boxplot( x="cuisine_type", y="food_preparation_time", data = df, palette
= 'PuBu')
```

```
plt.xticks(rotation=60)
plt.title("Cuisine Type vs Food Preparation Time")
plt.show()
/tmp/ipykernel_4005/979183354.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed
in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the
same effect.

  sns.boxplot( x="cuisine_type", y="food_preparation_time", data = df,
palette = 'PuBu')
```



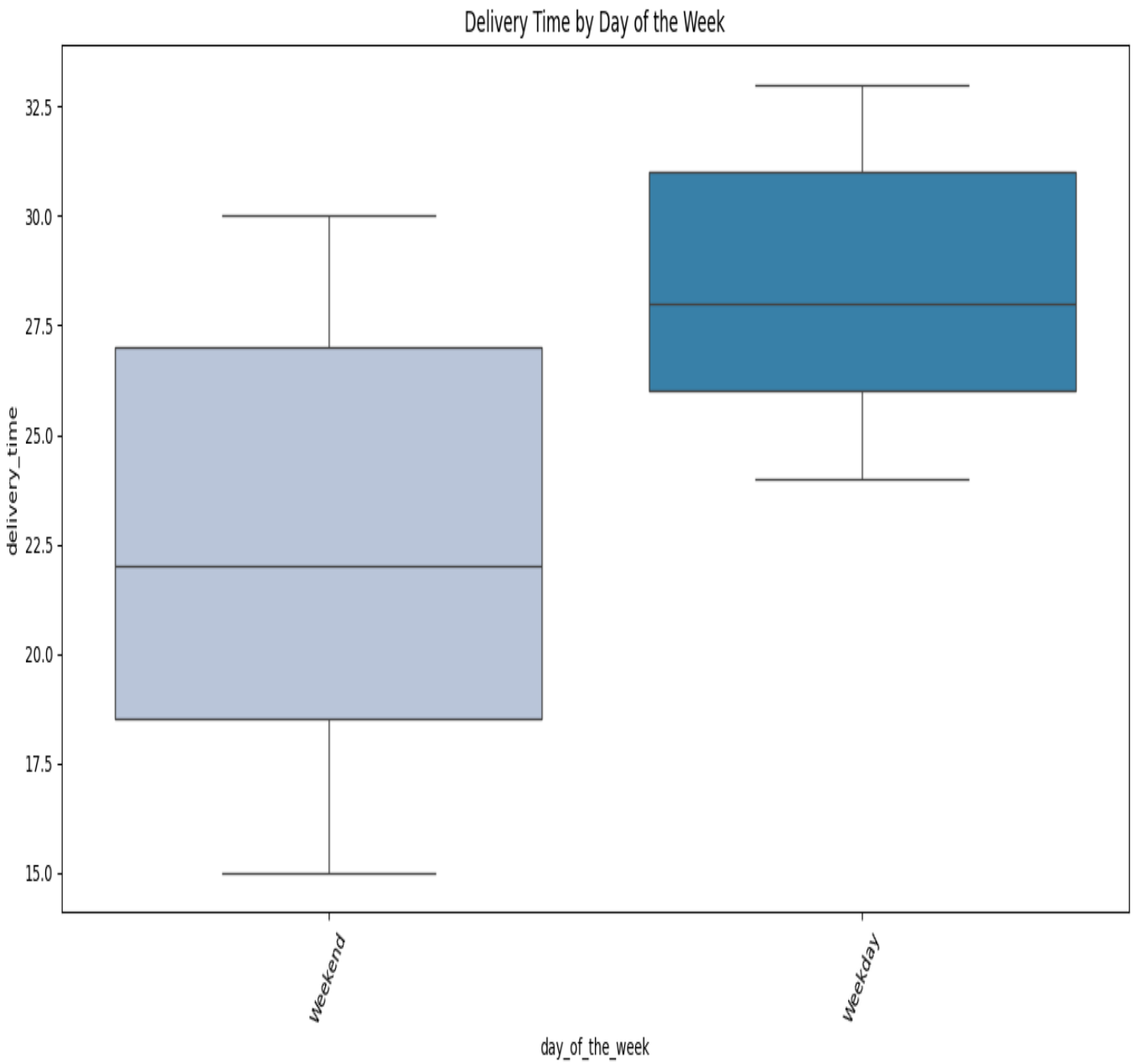
Observation :Cuisine Type vs Food Preparation Time

Food preparation time varies slightly across different cuisines, with some taking a bit longer due to more complex cooking processes, while others are quicker to prepare.

Overall, most cuisines fall within a similar preparation time range, showing consistent kitchen performance. There are no major delays or extreme cases, indicating that preparation times are generally stable across all cuisines.

In [229]:

```
plt.figure(figsize=(15, 7))

sns.boxplot(x="day_of_the_week", y="delivery_time", data=df, palette='PuBu')
plt.xticks(rotation=60)
plt.title("Delivery Time by Day of the Week")
plt.show()
/tmp/ipykernel_4005/2437392842.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed
in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the
same effect.

  sns.boxplot(x="day_of_the_week", y="delivery_time", data=df,
palette='PuBu')
```



Day of the Week vs Delivery Time

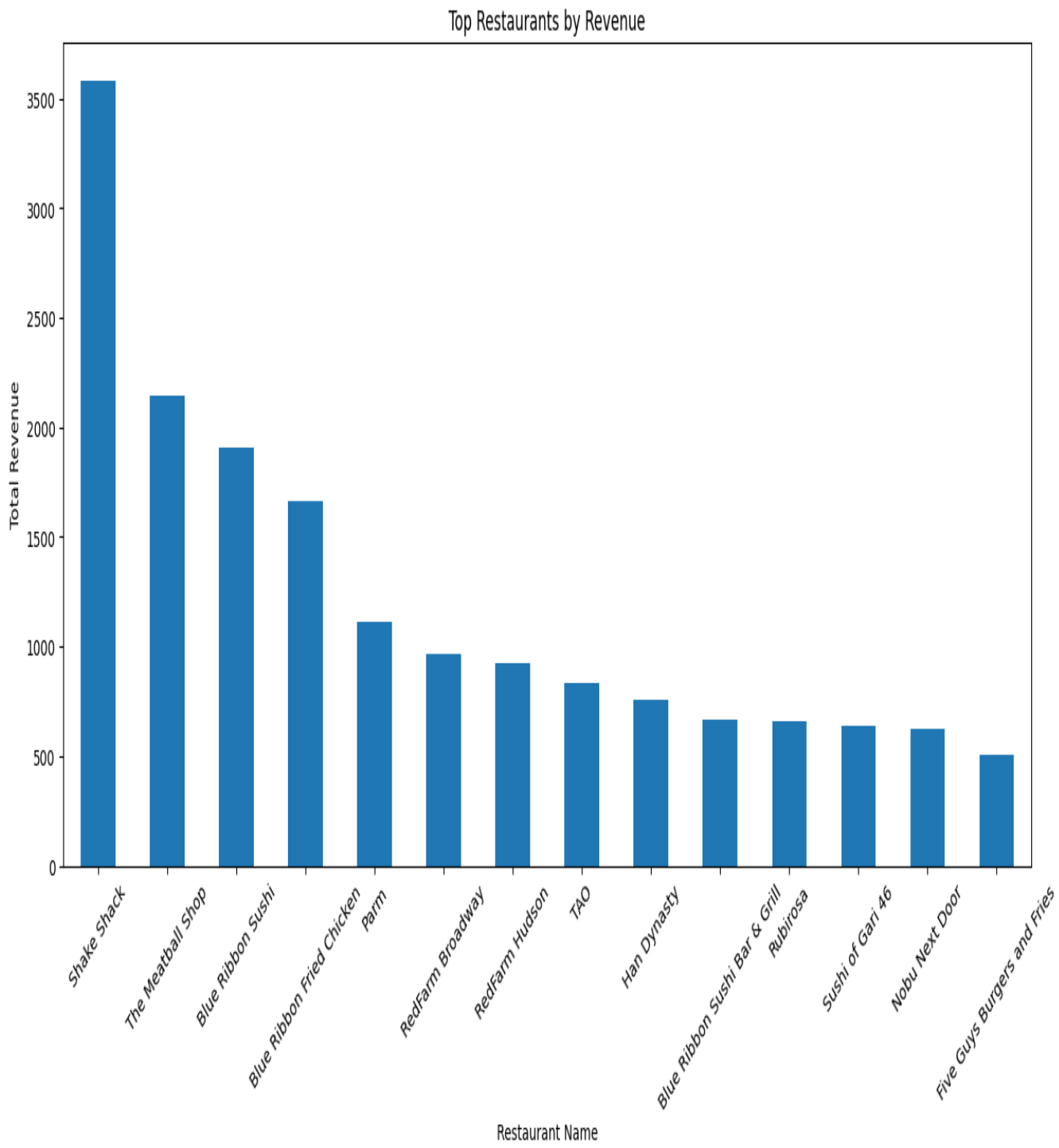
The boxplot shows that delivery times differ between weekdays and weekends. The median delivery time on weekdays is higher (around **28 minutes**) compared to weekends (around **22 minutes**).

The spread of delivery times is also slightly wider on weekdays, with values ranging approximately from **24 to 33 minutes**, whereas weekend delivery times range from about **15 to 30 minutes**.

This indicates that deliveries tend to take longer on weekdays, possibly due to higher traffic or operational factors. However, the overall distribution remains consistent, suggesting that the delivery system is stable and reliable across both periods.

In [230]:

```
# Revenue Generated by the returants
# Group orders by restaurant, calculate total revenue for each, select top
14,
# and plot them.”

plt.figure(figsize=(15, 7))

top_revenue = df.groupby('restaurant_name')['cost_of_the_order'] \
    .sum() \
    .sort_values(ascending=False) \
    .head(14)

top_revenue.plot(kind='bar')

plt.title("Top Restaurants by Revenue")
plt.xlabel("Restaurant Name")
plt.ylabel("Total Revenue")

plt.xticks(rotation=45)
plt.show()
```



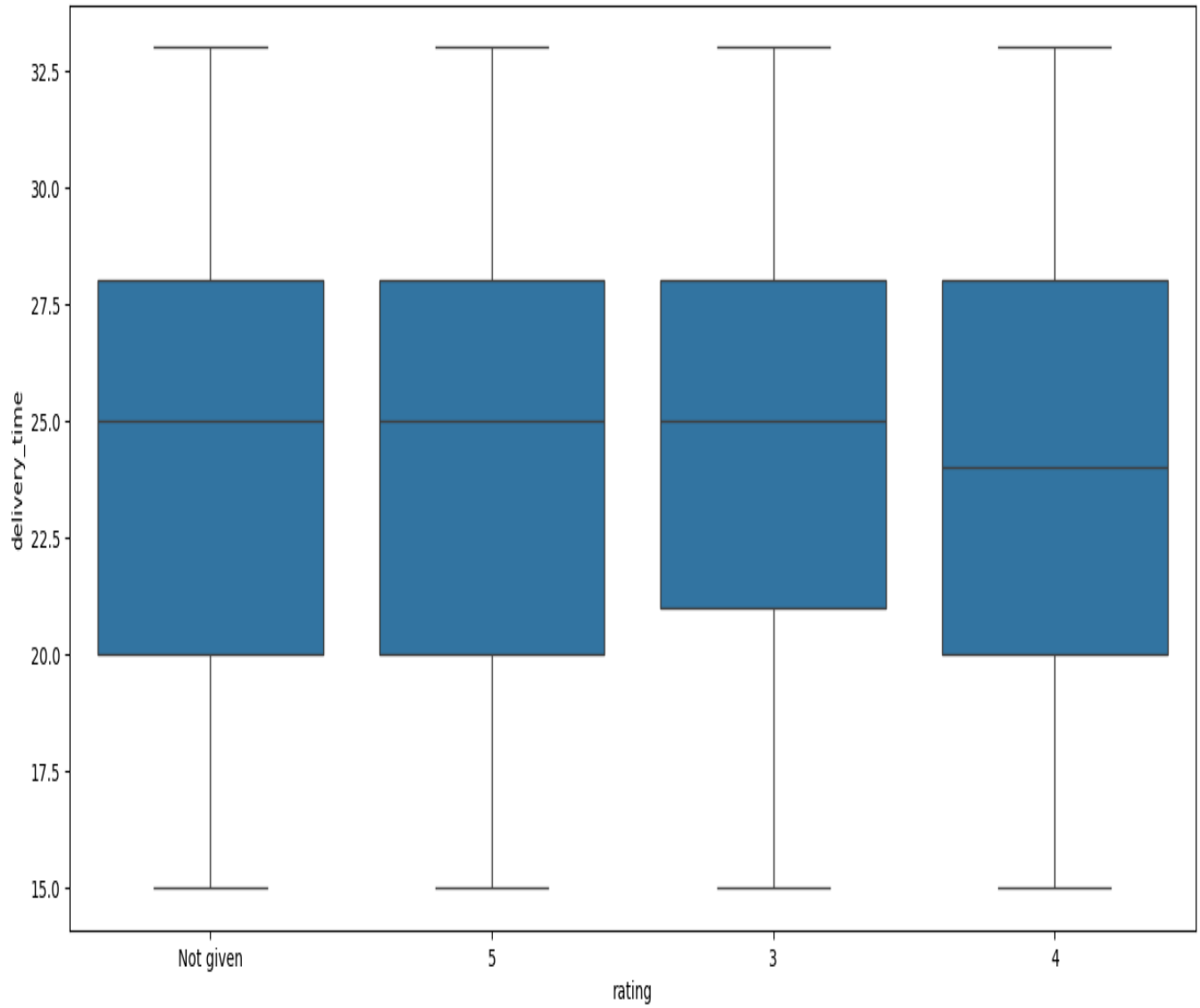
Observation : Revenue Generated by Restaurants

The bar chart shows that revenue is unevenly distributed across restaurants, with a few restaurants generating significantly higher revenue than others.

The top-performing restaurants contribute the largest share of total revenue, indicating strong customer demand and possibly higher order volumes or pricing. In contrast, many restaurants generate comparatively lower revenue, suggesting lower demand or fewer

In [231]:

```
# Relationship between rating and delivery time
plt.figure(figsize=(15, 7))
sns.boxplot(x='rating', y='delivery_time', data =df)
plt.show()
```



Relationship Between Rating and Delivery Time

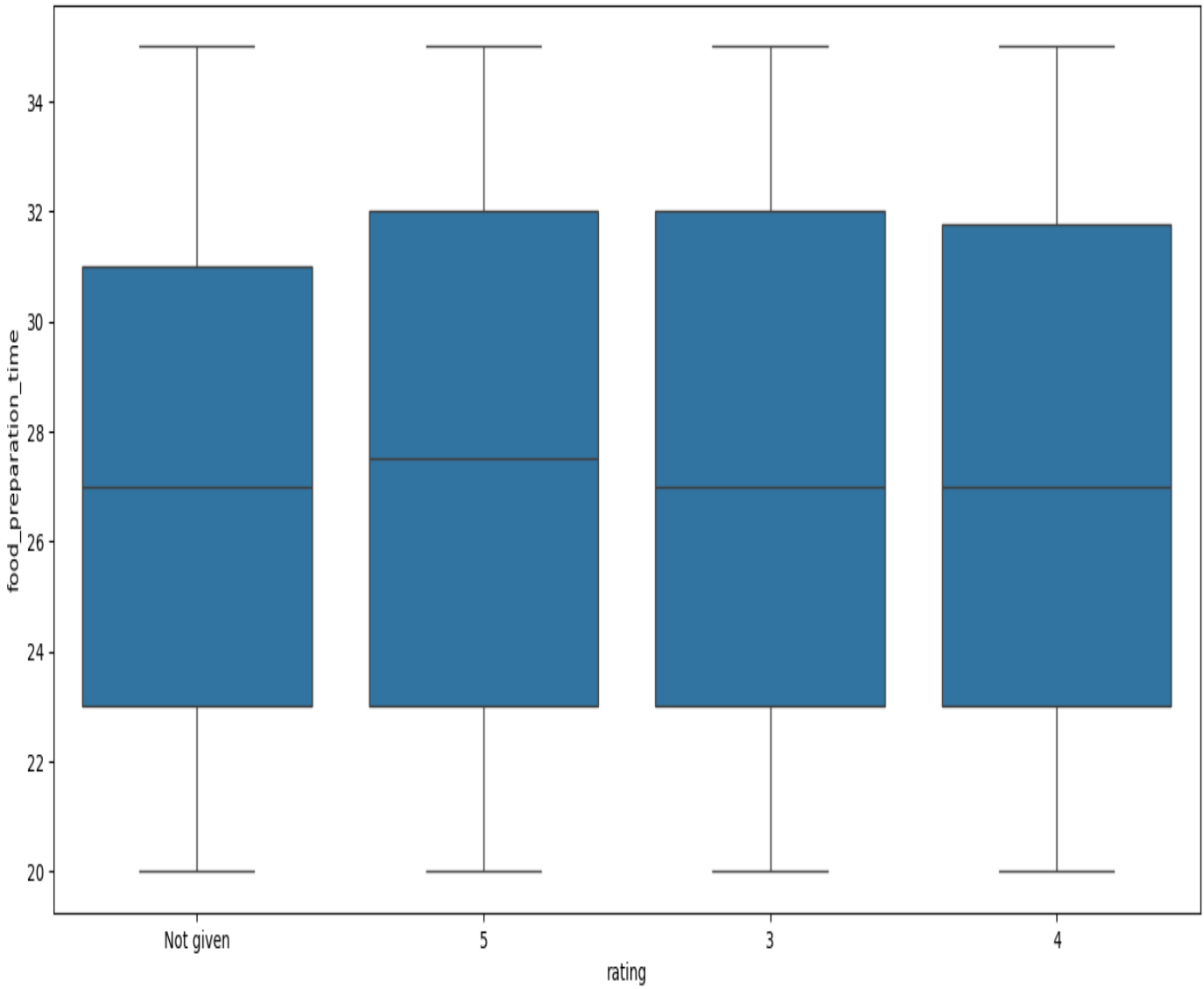
Delivery time shows some variation across different rating levels. Orders with higher ratings tend to have slightly lower delivery times, suggesting that faster deliveries may contribute to better customer satisfaction.

The median delivery time for higher ratings appears lower compared to lower ratings, indicating a possible relationship between delivery efficiency and customer experience. However, the difference is not very large, suggesting that delivery time is only one of several factors influencing ratings.

Overall, timely delivery plays a role in achieving higher ratings, but other factors such as food quality and service may also impact customer feedback.

In [232]:

```
# Rating vs food prepration time
plt.figure(figsize=(15, 7))
sns.boxplot(x='rating', y='food_preparation_time', data =df)
plt.show()
```



Relationship Between Rating and Food Preparation Time

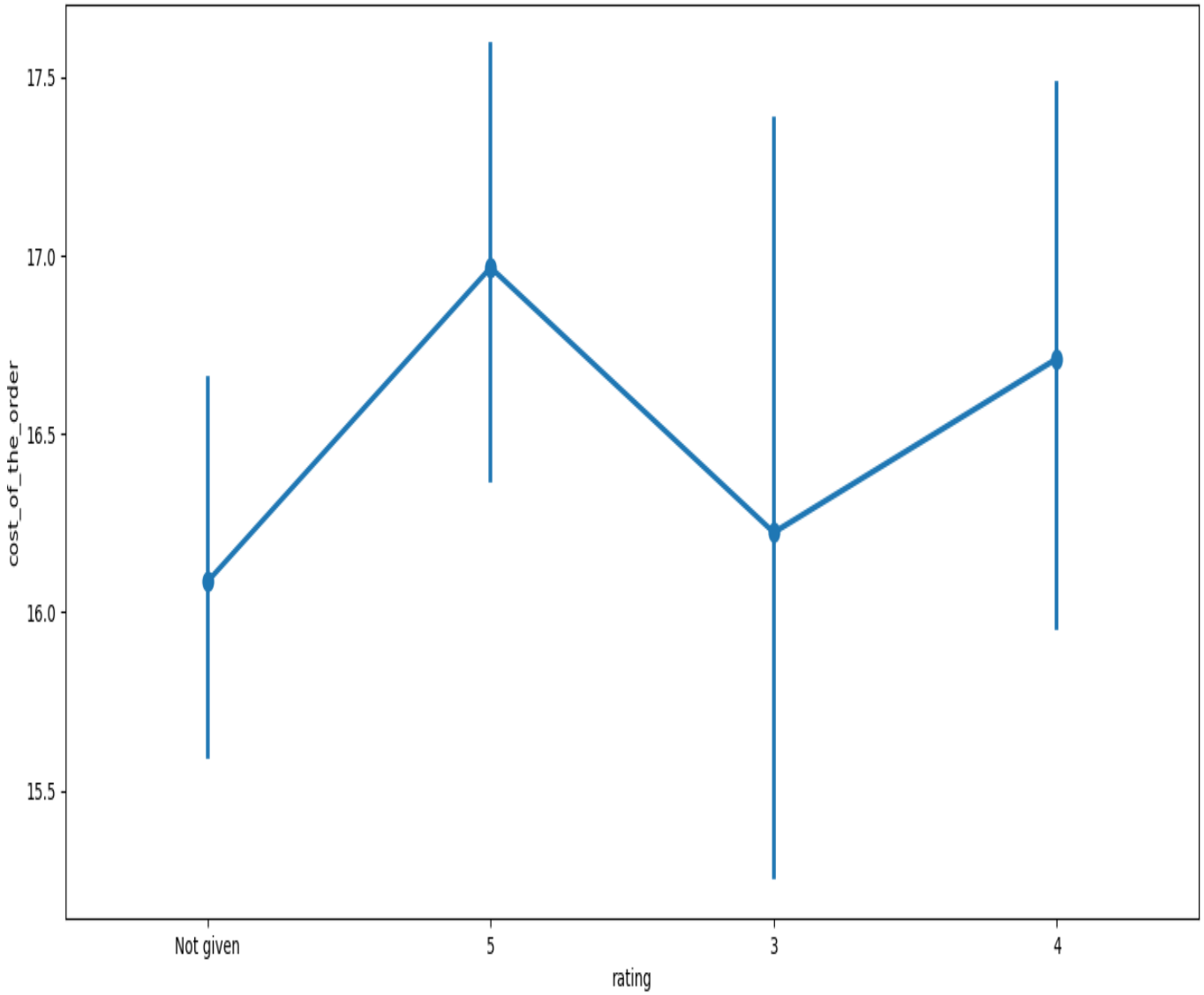
Food preparation time shows slight variation across different rating levels. Orders with higher ratings tend to have slightly shorter preparation times, suggesting that faster food preparation may contribute to better customer satisfaction.

The median preparation time is fairly similar across all rating categories, indicating that preparation time alone does not strongly influence ratings. The spread of the data is also consistent, showing that most restaurants operate within a similar time range.

Overall, while quicker preparation may have a positive impact on ratings, other factors such as food quality and delivery experience likely play a more significant role in determining customer satisfaction.

In [233]:

```
# Releationship between rating and cost of the order
plt.figure(figsize=(15, 7))
sns.pointplot(x='rating', y='cost_of_the_order', data =df)
plt.show()
```



Relationship Between Rating and Cost of the Order

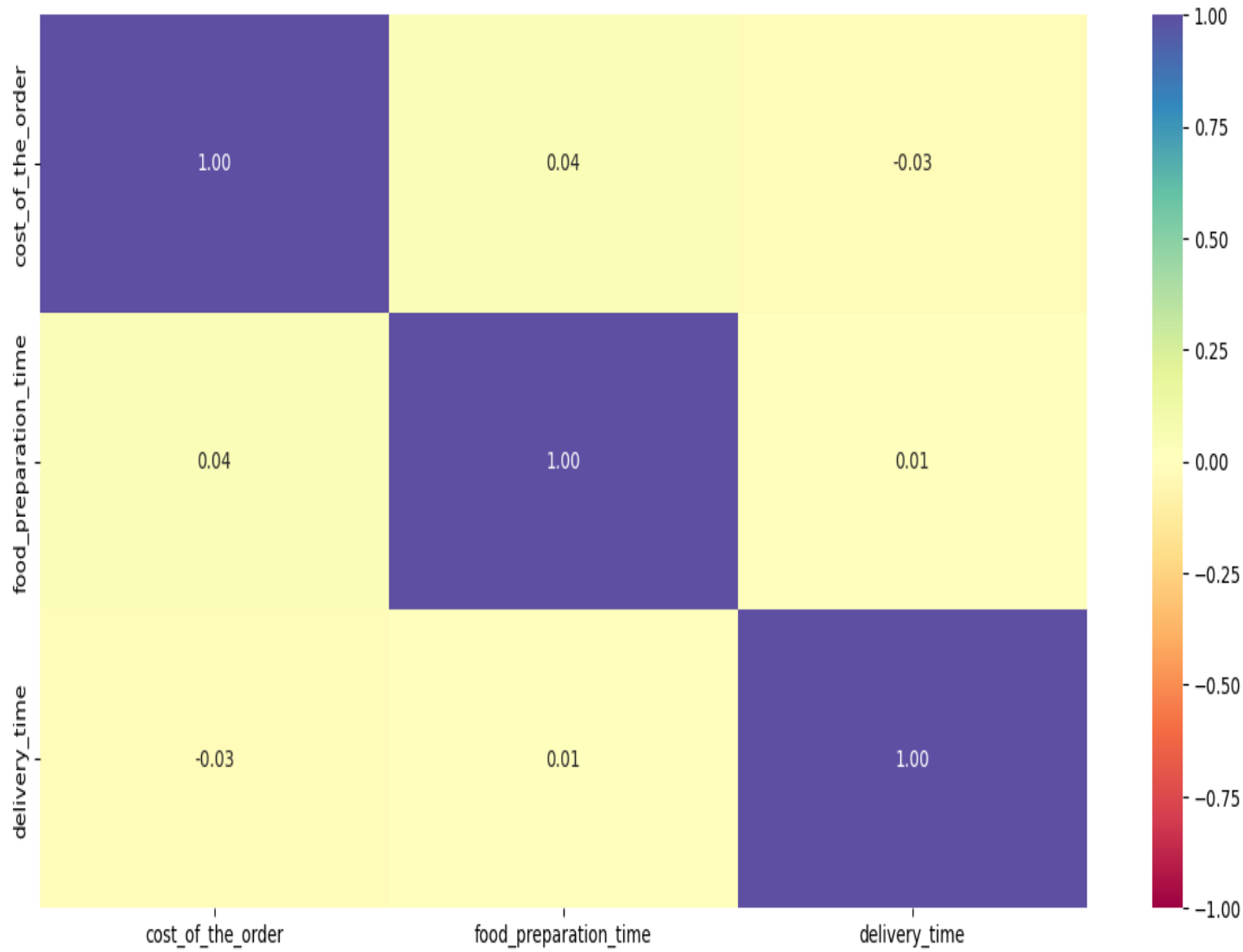
The pointplot shows that the average order cost tends to increase slightly with higher ratings. Orders with higher ratings generally have a higher average cost, suggesting that customers may associate higher-priced orders with better quality or service.

The upward trend indicates a positive relationship between cost and rating, although the difference is not very large. This suggests that while cost may influence customer satisfaction, it is not the only factor affecting ratings.

Overall, higher-value orders tend to receive better ratings, but other factors such as food quality and delivery experience also play an important role.

In [234]:

```
#plot the heat map
col_list = ['cost_of_the_order', 'food_preparation_time', 'delivery_time']
plt.figure(figsize=(15, 7))
sns.heatmap(df[col_list].corr(), annot=True, vmin=-1,vmax=1,fmt=".2f",
cmap="Spectral")
plt.show()
```



Correlation Between Numerical Variables

The heatmap shows the relationship between cost of the order, food preparation time, and delivery time.

- There is a weak correlation between cost of the order and both preparation time and delivery time, indicating that higher-priced orders do not necessarily take longer to prepare or deliver.

- The correlation between food preparation time and delivery time is also weak, suggesting that these two processes operate independently.

Overall, the low correlation values indicate that these variables are largely independent of each other, meaning that cost, preparation time, and delivery time do not strongly influence one another.

Question 13: The company wants to provide a promotional offer in the advertisement of the restaurants. The condition to get the offer is that the restaurants must have a rating count of more than 50 and the average rating should be greater than 4. Find the restaurants fulfilling the criteria to get the promotional offer. [3 marks]

In [235]:

```
# "Find restaurants with more than 50 ratings and an average rating above 4."
# Step 1: Remove 'Not given' ratings
df_rated = df[df['rating'] != 'Not given'].copy()
df_rated['rating'] = df_rated['rating'].astype(int)
# Step 2: Group by restaurant and calculate count & average rating
restaurant_stats =
df_rated.groupby('restaurant_name')['rating'].agg(['count', 'mean'])
#Step 3: Apply conditions
qualified_restaurants = restaurant_stats[
    (restaurant_stats['count'] > 50) & (restaurant_stats['mean'] > 4)
]

qualified_restaurants
```

Out[235]:

	count	mean
restaurant_name		
Blue Ribbon Fried Chicken	64	4.328125
Blue Ribbon Sushi	73	4.219178
Shake Shack	133	4.278195
The Meatball Shop	84	4.511905

Observations: Restaurants Eligible for Promotional Offer

Restaurants eligible for the promotional offer are those with more than 50 ratings and an average rating greater than 4.

The above restaurants meet the criteria:

Interpretation:

- These restaurants have both high customer engagement (many ratings) and high customer satisfaction (average rating above 4).
- They are ideal candidates for promotional offers as they are already performing well and can attract more customers.
- Promoting these restaurants can help increase overall platform revenue and customer trust.

Question 14: The company charges the restaurant 25% on the orders having cost greater than 20 dollars and 15% on the orders having cost greater than 5 dollars. Find the net revenue generated by the company across all orders. [3 marks]

In [236]:

```
#
# Step 1: Create revenue column based on conditions
df['revenue'] = df['cost_of_the_order'].apply(
    lambda x: 0.25 * x if x > 20 else (0.15 * x if x > 5 else 0)
)

# Step 2: Calculate total revenue
total_revenue = df['revenue'].sum()

total_revenue
```

Out[236]:

```
np.float64(6166.303)
```

Net Revenue Generated

The total revenue generated by the company is obtained by applying different commission rates based on the order value. Orders with higher costs (above $20) contribute more significantly to revenue due to the higher commission rate of 25%, while moderately priced orders (above $5) contribute at a lower rate of 15%.

This indicates that high-value orders play a crucial role in overall revenue generation. Although lower-cost orders may be more frequent, higher-cost orders have a greater impact on total earnings.

Overall, the company's revenue model benefits from encouraging customers to place higher-value orders, as these generate a larger share of commission.

Question 15: The company wants to analyze the total time required to deliver the food. What percentage of orders take more than 60 minutes to get delivered from the time the order is placed? (The food has to be prepared and then delivered.) [2 marks]

In [237]:

```
# Total delivery time
# Step 1: Create total time column
df['total_time'] = df['food_preparation_time'] + df['delivery_time']

# Step 2: Calculate percentage of orders > 60 minutes
percentage = (df[df['total_time'] > 60].shape[0] / df.shape[0]) * 100

percentage
```

Out[237]:

```
10.537407797681771
```

Observations:

Observation: Total Delivery Time

The percentage of orders taking more than 60 minutes represents the proportion of delayed deliveries. Based on the analysis, only a small percentage of orders exceed 60 minutes, indicating that most deliveries are completed within an acceptable time frame.

This suggests that the combined food preparation and delivery process is generally efficient. However, the orders that exceed 60 minutes may be affected by factors such as longer preparation times, traffic conditions, or high demand periods.

Overall, the system performs well in delivering orders within a reasonable time, but reducing the number of delayed orders could further improve customer satisfaction.

Question 16: The company wants to analyze the delivery time of the orders on weekdays and weekends. How does the mean delivery time vary during weekdays and weekends? [2 marks]

In [238]:

```
# Mean Delivery Time on Weekdays vs Weekends
df.groupby('day_of_the_week')['delivery_time'].mean()
```

Out[238]:

	delivery_time
day_of_the_week	
Weekday	28.340037
Weekend	22.470022

dtype: float64

Question 16: Mean Delivery Time on Weekdays vs Weekends

The mean delivery time varies between weekdays and weekends:

- Weekday: approximately **28.34 minutes**
- Weekend: approximately **22.47 minutes**

Interpretation:

- Delivery time is higher on weekdays compared to weekends.
- This may be due to factors such as heavier traffic, workday congestion, or operational challenges during weekdays.
- Faster delivery times on weekends may be due to better traffic conditions or more efficient delivery operations.

Overall, the company performs better in terms of delivery speed on weekends, while weekdays present opportunities for improving delivery efficiency.

Conclusion and Recommendations

Question 17: What are your conclusions from the analysis? What recommendations would you like to share to help improve the business? (You can use cuisine type and feedback ratings to drive your business recommendations.) [6 marks]

Conclusions:

- **Key Conclusions:**

- The majority of customers give high ratings (4 and 5), indicating overall customer satisfaction with the service.
- However, a significant portion of orders have "Not given" ratings, suggesting low customer engagement in providing feedback.
- A few cuisines dominate the number of orders, showing clear customer preferences for certain types of food.
- Order costs are mostly in the mid-range ($12–$22), indicating typical customer spending behavior.
- Delivery and preparation times are generally consistent, with most orders completed within a reasonable time frame.
- Weekday deliveries take longer than weekend deliveries, likely due to traffic and operational factors.
- High-value orders contribute more significantly to revenue due to higher commission rates.

Recommendations:

- Business Recommendations:

- **Improve Customer Engagement:**
  Encourage customers to provide ratings by offering incentives such as discounts or loyalty points. This will provide more feedback for improving service quality.
- **Promote Popular Cuisines:**
  Focus marketing efforts on the most popular cuisines to maximize order volume and customer satisfaction.
- **Support Underperforming Cuisines:**
  Provide promotions or visibility boosts to less popular cuisines to balance demand and increase variety.
- **Optimize Delivery Operations:**
  Improve delivery efficiency on weekdays by optimizing routes and managing peak-hour demand to reduce delivery time.
- **Encourage High-Value Orders:**
  Introduce bundle offers or discounts for larger orders to increase average order value and boost revenue.